

Vamos fazer um exemplo **bem simples, do zero**, usando Spring Boot + Spring Security para entender como **proteger endpoints**.

A ideia será:

- /publico → qualquer pessoa pode acessar
- /produtos → precisa estar autenticado
- /admin → precisa estar autenticado e ter perfil ADMIN
- usuário e senha ficarão **em memória**, para facilitar o aprendizado

1. Dependência

No `pom.xml`, adicione:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Se você já tem `spring-boot-starter-web`, ficará basicamente assim:

```
<dependencies>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

</dependencies>
```

2. Crie um Controller

Vamos criar:

`ProdutoController.java`

```
package com.exemplo.demo.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController  
public class ProdutoController {  
  
    @GetMapping("/publico")  
    public String publico() {  
        return "Endpoint público";  
    }  
  
    @GetMapping("/produtos")  
    public String produtos() {  
        return "Lista de produtos";  
    }  
  
    @GetMapping("/admin")  
    public String admin() {  
        return "Área administrativa";  
    }  
}
```

Temos três endpoints:

Endpoint	Acesso
/publico	Todos
/produtos	Usuário autenticado
/admin	Usuário com perfil ADMIN

3. Criar a configuração do Spring Security

Crie:

SecurityConfig.java

```
package com.exemplo.demo.config;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

@Configuration

```
public class SecurityConfig {
```

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {
```

```
    http
```

```
        .authorizeHttpRequests(auth -> auth
```

```
            .requestMatchers("/publico").permitAll()
```

```
            .requestMatchers("/admin").hasRole("ADMIN")
```

```
            .requestMatchers("/produtos").authenticated()
```

```
            .anyRequest().authenticated()
```

```
        )
```

```
        .httpBasic();
```

```
    return http.build();
```

```
}
```

@Bean

```
public InMemoryUserDetailsManager usuarios() {
```

```
    UserDetails usuario = User
```

```
        .withUsername("aluno")
```

```
        .password("{noop}123")
```

```
        .roles("USER")
```

```
        .build();
```

```
    UserDetails administrador = User
```

```
        .withUsername("admin")
```

```
        .password("{noop}123")
```

```
        .roles("ADMIN")
```

```

        .build();

        return new InMemoryUserDetailsManager(
            usuario,
            administrador
        );
    }
}

```

Agora vamos entender **cada parte**.

4. O que é SecurityFilterChain?

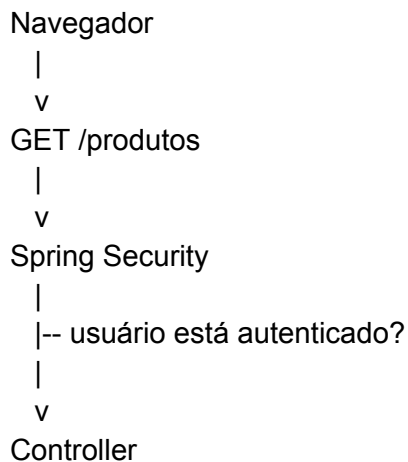
Esta é a parte mais importante:

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

```

O Spring Security funciona principalmente através de **filtros**. Imagine que uma requisição chega:



O Spring Security verifica a requisição **antes de ela chegar ao Controller**.

5. Liberando um endpoint

Temos:

```
.requestMatchers("/publico").permitAll()
```

`permitAll()` significa:

Qualquer pessoa pode acessar.

Então:

GET /publico

não exige login.

6. Exigindo autenticação

Agora:

```
.requestMatchers("/produtos").authenticated()
```

`authenticated()` significa:

O usuário precisa estar autenticado.

Se alguém tentar:

GET /produtos

sem informar usuário e senha, o Spring Security bloqueará o acesso.

7. Protegendo por perfil

Temos:

```
.requestMatchers("/admin").hasRole("ADMIN")
```

Isso é muito importante.

Significa:

Para acessar /admin, o usuário precisa ter o papel ADMIN.

Temos dois usuários:

```
UserDetails usuario = User
    .withUsername("aluno")
    .password("{noop}123")
    .roles("USER")
    .build();
```

Esse usuário possui:

usuário: aluno
senha: 123
perfil: USER

E:

```
UserDetails administrador = User
    .withUsername("admin")
    .password("{noop}123")
    .roles("ADMIN")
    .build();
```

Possui:

usuário: admin
senha: 123
perfil: ADMIN

8. O que acontece na prática?

Imagine que o aluno tente acessar:

/produtos

Ele fornece:

aluno
123

O Spring Security verifica:

Usuário existe?

```

|
v
SIM
|
v
Senha correta?
|
v
SIM
|
v
Está autenticado?
|
v
SIM
|
v
Permitir /produtos

```

Então o Controller executa:

```

@GetMapping("/produtos")
public String produtos() {
    return "Lista de produtos";
}

```

9. E se o aluno tentar /admin?

Ele está autenticado, mas possui:

```
ROLE_USER
```

Enquanto o endpoint exige:

```
ROLE_ADMIN
```

Então:

```

aluno
|
| ROLE_USER
v
/admin

```

|
| precisa ROLE_ADMIN
v
ACESSO NEGADO

Normalmente será retornado:

403 Forbidden

10. O administrador consegue acessar

O administrador utiliza:

usuário: admin
senha: 123

Ele possui:

ROLE_ADMIN

Então:

admin
|
| ROLE_ADMIN
v
/admin
|
v
PERMITIDO

E receberá:

Área administrativa

11. Por que usamos {noop}?

No exemplo:

.password("{noop}123")

Estamos dizendo ao Spring Security:

Essa senha está armazenada sem criptografia, apenas para este exemplo.

Não faça isso em uma aplicação real.

Em uma aplicação real, devemos utilizar um PasswordEncoder, por exemplo BCrypt:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

E:

```
.password(passwordEncoder().encode("123"))
```

Para alunos iniciantes, eu sugiro primeiro entender o funcionamento com {noop} e depois passar para BCrypt.

12. E o .anyRequest()?

Temos:

```
.anyRequest().authenticated()
```

Isso significa:

Qualquer outro endpoint que não foi especificado anteriormente também precisa de autenticação.

Por exemplo, se você criar:

```
@GetMapping("/clientes")
public String clientes() {
    return "Lista de clientes";
}
```

Como não especificamos /clientes, ele cairá em:

```
.anyRequest().authenticated()
```

Portanto:

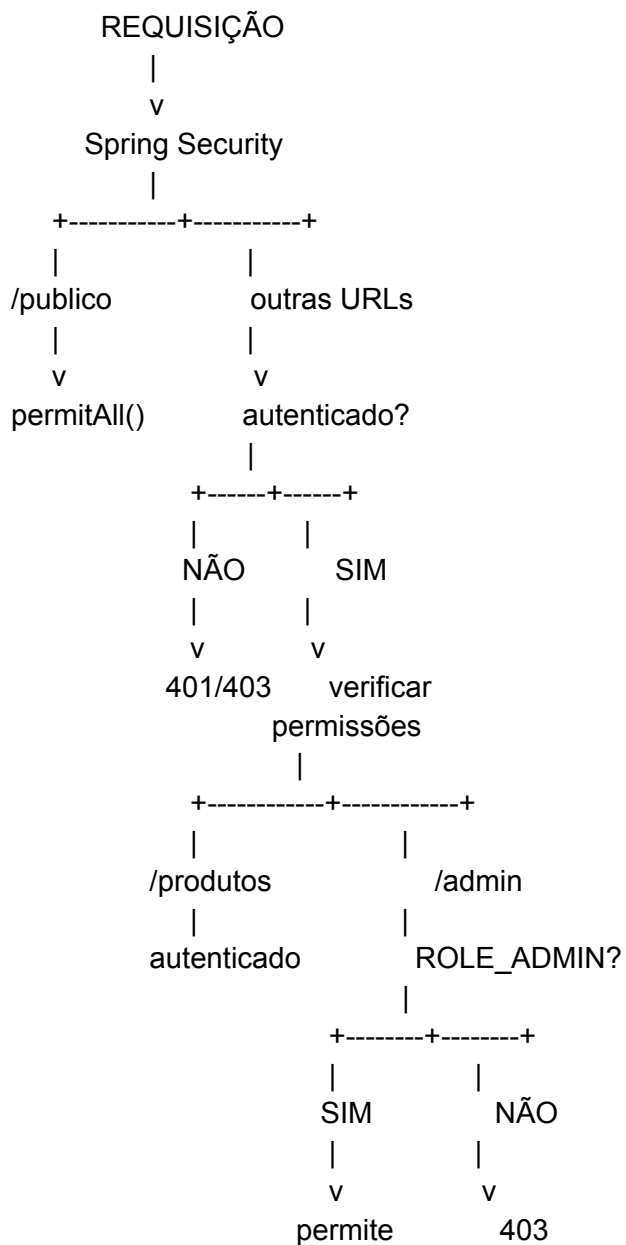
GET /clientes



Precisa estar autenticado

13. A regra de segurança fica assim

Podemos visualizar:



14. Uma observação importante sobre hasRole

Quando escrevemos:

```
.roles("ADMIN")
```

o Spring Security internamente trabalha com:

```
ROLE_ADMIN
```

Por isso escrevemos:

```
.hasRole("ADMIN")
```

e **não**:

```
.hasRole("ROLE_ADMIN")
```

15. Testando

Execute:

```
mvn spring-boot:run
```

Depois acesse:

```
http://localhost:8080/publico
```

Deve aparecer: Endpoint público

Acesse: <http://localhost:8080/produtos>

Será solicitado usuário e senha.

Use:

```
aluno  
123
```

Depois teste:

http://localhost:8080/admin

Com:

aluno
123

O acesso será negado.

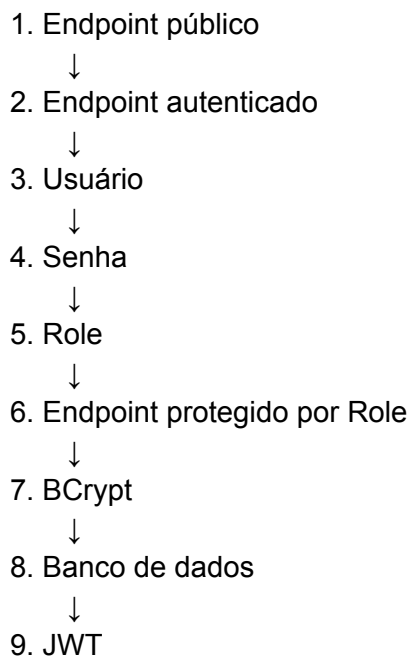
Depois utilize:

admin
123

e o acesso será permitido.

16. O conceito principal

Spring Security inicialmente com esta sequência:



O ponto fundamental é separar dois conceitos:

Autenticação

"Quem é você?"

```
.authenticated()
```

Autorização

"Você tem permissão para fazer isso?"

```
.hasRole("ADMIN")
```

Assim, o aluno consegue entender primeiro **o mecanismo de proteção dos endpoints** antes de entrar em JWT, banco de dados e autenticação mais avançada.

Mostre agora um projeto com jwt acessando banco de dados

Sim. Vamos evoluir o exemplo anterior para um projeto **Spring Boot + Spring Security + JWT + MySQL**, usando um CRUD simples de produtos.

A arquitetura será:

Cliente

|
| POST /auth/login
v

Spring Boot

|
v

MySQL

|
| usuário/senha
v

JWT

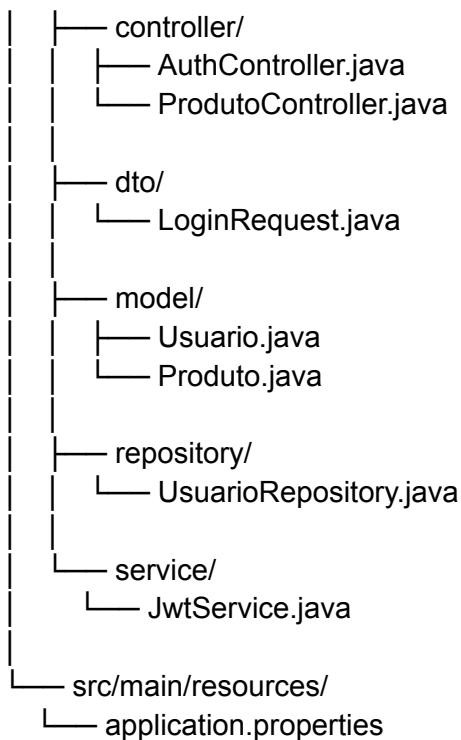
|
| Authorization: Bearer TOKEN
v

Endpoints protegidos

Vou montar de forma didática, pensando em **alunos iniciantes**.

1. Estrutura do projeto

```
spring-jwt/  
├── src/main/java/com/exemplo/demo/  
│   ├── DemoApplication.java  
│   ├── config/  
│   │   └── SecurityConfig.java
```



2. Dependências

No pom.xml:

```
<dependencies>
```

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
```

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
```

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
```

```
<dependency>
```

```
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
</dependencies>
```

Aqui temos quatro partes importantes:

Spring Web



cria a API REST

Spring Data JPA



acessa o banco

Spring Security



protege os endpoints

JJWT



cria e valida os tokens JWT

3. Banco de dados

No MySQL:

```
CREATE DATABASE loja;
```

```
USE loja;
```

A tabela de usuários será criada pelo JPA.

Vamos inicialmente ter:

```
usuario
```

```
-----
```

```
id
```

```
nome
```

```
username
```

```
password
```

```
role
```

4. Configuração do banco

`application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/loja
```

```
spring.datasource.username=root
```

```
spring.datasource.password=123456
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

```
server.port=8080
```

Altere:

```
spring.datasource.username=root
```

```
spring.datasource.password=123456
```

para os dados do seu MySQL.

5. Criando a entidade Usuario

Usuario.java

```
package com.exemplo.demo.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "usuarios")
```

```
public class Usuario {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String nome;
```

```
    @Column(unique = true)
```

```
    private String username;
```

```
    private String password;
```

```
    private String role;
```

```
    public Usuario() {
```

```
    }
```

```
    public Usuario(String nome, String username,
```

```
        String password, String role) {
```

```
        this.nome = nome;
```

```
        this.username = username;
```

```
        this.password = password;
```

```
        this.role = role;
```

```
    }
```

```
    public Long getId() {
```

```
        return id;
```

```
    }
```

```
    public String getNome() {
```

```
        return nome;
```

```
    }
```

```
public String getUsername() {  
    return username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public String getRole() {  
    return role;  
}  
  
public void setId(Long id) {  
    this.id = id;  
}  
  
public void setName(String nome) {  
    this.nome = nome;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public void setRole(String role) {  
    this.role = role;  
}  
}
```

6. Repository

Agora precisamos conseguir procurar um usuário pelo username.

UsuarioRepository.java

```
package com.exemplo.demo.repository;
```

```
import com.exemplo.demo.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(String username);
}
```

O Spring Data JPA implementará automaticamente:

```
findByUsername(...)
```

Então podemos fazer:

```
usuarioRepository.findByUsername("admin");
```

e o Spring fará algo equivalente a:

```
SELECT *
FROM usuarios
WHERE username = 'admin';
```

7. DTO para login

Crie:

```
LoginRequest.java
```

```
package com.exemplo.demo.dto;
```

```
public class LoginRequest {

    private String username;
    private String password;

    public String getUsername() {
        return username;
    }

    public String getPassword() {
```

```
        return password;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Esse objeto receberá:

```
{
  "username": "admin",
  "password": "123456"
}
```

8. Criando o JwtService

Agora vem uma das partes principais.

JwtService.java

```
package com.exemplo.demo.service;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;

@Service
public class JwtService {

    private final String SECRET =
        "minha-chave-secreta-muito-grande-para-jwt-123456";

    private final long EXPIRATION = 1000 * 60 * 60;
```

```
private SecretKey getKey() {
    return Keys.hmacShaKeyFor(
        SECRET.getBytes()
    );
}

public String gerarToken(String username) {

    return Jwts.builder()
        .subject(username)
        .issuedAt(new Date())
        .expiration(
            new Date(
                System.currentTimeMillis()
                + EXPIRATION
            )
        )
        .signWith(getKey())
        .compact();
}

public String extrairUsername(String token) {

    return Jwts.parser()
        .verifyWith(getKey())
        .build()
        .parseSignedClaims(token)
        .getPayload()
        .getSubject();
}
}
```

9. Entendendo o JWT

Quando fazemos:

```
gerarToken("admin")
```

o servidor gera algo parecido com:

`eyJhbGciOiJIUzI1NiJ9.eyJzdWwiOiJhZG1pbjE5LmVycy5kYXRlLnRlcmlkaXQxMDE0In0.eYJhbGciOiJIUzI1NiJ9`

O token possui três partes:

HEADER.PAYLOAD.SIGNATURE

Por exemplo:

HEADER

|
v
algoritmo utilizado

PAYLOAD

|
v
username
data de criação
data de expiração

SIGNATURE

|
v
assinatura digital

Importante: JWT não significa que os dados do payload estejam criptografados. Eles são codificados e assinados. Portanto, não coloque senha ou informações sensíveis no payload.

10. Criando o Controller de autenticação

AuthController.java

```
package com.exemplo.demo.controller;
```

```
import com.exemplo.demo.dto.LoginRequest;  
import com.exemplo.demo.model.Usuario;  
import com.exemplo.demo.repository.UsuarioRepository;  
import com.exemplo.demo.service.JwtService;
```

```
import org.springframework.http.ResponseEntity;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.Map;
```

```

@RestController
@RequestMapping("/auth")
public class AuthController {

    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public AuthController(
        UsuarioRepository usuarioRepository,
        PasswordEncoder passwordEncoder,
        JwtService jwtService) {

        this.usuarioRepository = usuarioRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(
        @RequestBody LoginRequest request) {

        Usuario usuario =
            usuarioRepository
                .findByUsername(request.getUsername())
                .orElse(null);

        if (usuario == null) {
            return ResponseEntity
                .status(401)
                .body("Usuário ou senha inválidos");
        }

        if (!passwordEncoder.matches(
            request.getPassword(),
            usuario.getPassword())) {

            return ResponseEntity
                .status(401)
                .body("Usuário ou senha inválidos");
        }

        String token =

```

```

        jwtService.gerarToken(
            usuario.getUsername());

        return ResponseEntity.ok(
            Map.of("token", token)
        );
    }
}

```

Aqui acontece o processo fundamental:

```

POST /auth/login
  |
  v
username + password
  |
  v
Consulta banco
  |
  v
Usuário encontrado?
  |
  v
Confere senha
  |
  v
Senha correta?
  |
  v
Gera JWT
  |
  v
Retorna token

```

11. PasswordEncoder

Na configuração do Security:

```

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

```


O BCrypt transforma:

123456

em algo parecido com:

\$2a\$10\$xx

Assim, **a senha real não fica armazenada no banco.**

12. SecurityConfig

Agora vamos configurar o Spring Security.

SecurityConfig.java

```
package com.exemplo.demo.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
```

```
@Configuration
```

```
public class SecurityConfig {
```

```
    @Bean
```

```
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {
```

```
        http
```

```
            .csrf(csrf -> csrf.disable())
```

```
            .sessionManagement(session ->
```

```
                session.sessionCreationPolicy(
```

```
                    SessionCreationPolicy.STATELESS
```

```
                )
```

```
            )
```

```

        .authorizeHttpRequests(auth -> auth

            .requestMatchers(
                "/auth/**"
            ).permitAll()

            .requestMatchers(
                "/produtos/**"
            ).authenticated()

            .anyRequest().authenticated()
        );

    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {

    return new BCryptPasswordEncoder();
}
}

```

Observe:

```

.requestMatchers("/auth/**")
.permitAll()

```

Login não precisa de token.

Enquanto:

```

.requestMatchers("/produtos/**")
.authenticated()

```

exige autenticação.

13. Ainda falta uma peça

Até aqui temos:

login

↓
banco
↓
JWT

Mas quando o usuário acessar:

GET /produtos

precisamos pegar:

Authorization: Bearer TOKEN

e verificar esse token.

Para isso criamos um **JWT Filter**.

14. JwtAuthenticationFilter

Crie:

`JwtAuthenticationFilter.java`

```
package com.exemplo.demo.config;
```

```
import com.exemplo.demo.service.JwtService;
```

```
import jakarta.servlet.FilterChain;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
```

```
import org.springframework.security.core.context.SecurityContextHolder;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import java.io.IOException;
```

```
import java.util.Collections;
```

```
@Component
```

```
public class JwtAuthenticationFilter
```

```

    extends OncePerRequestFilter {

private final JwtService jwtService;

public JwtAuthenticationFilter(
    JwtService jwtService) {

    this.jwtService = jwtService;
}

@Override
protected void doFilterInternal(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain filterChain)
    throws ServletException, IOException {

    String header =
        request.getHeader("Authorization");

    if (header == null ||
        !header.startsWith("Bearer ")) {

        filterChain.doFilter(request, response);
        return;
    }

    String token =
        header.substring(7);

    try {

        String username =
            jwtService.extractUsername(token);

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                username,
                null,
                Collections.emptyList()
            );

        SecurityContextHolder
            .getContext()

```

```
        .setAuthentication(authentication);

    } catch (Exception e) {

        response.setStatus(
            HttpServletResponse.SC_UNAUTHORIZED
        );

        return;
    }

    filterChain.doFilter(request, response);
}
}
```

15. Registrando o Filter

Precisamos modificar o SecurityConfig.

Adicione:

```
private final JwtAuthenticationFilter jwtAuthenticationFilter;
```

E o construtor:

```
public SecurityConfig(
    JwtAuthenticationFilter jwtAuthenticationFilter) {

    this.jwtAuthenticationFilter =
        jwtAuthenticationFilter;
}
```

Depois:

```
.addFilterBefore(
    jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class
)
```

O código completo fica:

```
package com.exemplo.demo.config;
```

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

```

@Configuration

```

public class SecurityConfig {

```

```

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

```

```

    public SecurityConfig(
        JwtAuthenticationFilter jwtAuthenticationFilter) {

```

```

        this.jwtAuthenticationFilter =
            jwtAuthenticationFilter;

```

```

    }

```

@Bean

```

    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

```

```

        http
            .csrf(csrf -> csrf.disable())

            .sessionManagement(session ->
                session.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS
                )
            )

```

```

        .authorizeHttpRequests(auth -> auth

```

```

            .requestMatchers("/auth/**")
            .permitAll()

```

```

            .requestMatchers("/produtos/**")
            .authenticated()

```

```

            .anyRequest()
            .authenticated()

```

```

    )

    .addFilterBefore(
        jwtAuthenticationFilter,
        UsernamePasswordAuthenticationFilter.class
    );

    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

16. Criando o endpoint protegido

ProdutoController.java

```

package com.exemplo.demo.controller;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    @GetMapping
    public String listar() {

        return "Lista de produtos";
    }

    @GetMapping("/{id}")
    public String consultar(
        @PathVariable Long id) {

        return "Produto " + id;
    }

    @PostMapping

```

```
public String salvar() {  
    return "Produto cadastrado";  
}  
}
```

Agora:

GET /produtos

é protegido.

17. Criando o primeiro usuário

Precisamos colocar um usuário no banco.

Uma forma didática é criar um CommandLineRunner.

Na classe principal:

```
@Bean  
CommandLineRunner criarUsuario(  
    UsuarioRepository repository,  
    PasswordEncoder encoder) {  
  
    return args -> {  
  
        if (repository.findByUsername("admin").isEmpty()) {  
  
            Usuario usuario = new Usuario();  
  
            usuario.setNome("Administrador");  
            usuario.setUsername("admin");  
  
            usuario.setPassword(  
                encoder.encode("123456")  
            );  
  
            usuario.setRole("ADMIN");  
  
            repository.save(usuario);  
        }  
    }  
}
```



```
};  
}
```

Quando a aplicação iniciar, será criado:

```
username: admin  
password: 123456  
role: ADMIN
```

No banco teremos algo parecido com:

id	nome	username	password	role
1	Administrador	admin	\$2a\$10\$xxxxxxxxxxxxx...	ADMIN

Observe que a senha **não é 123456 no banco**.

18. Testando o login

Use Postman, Insomnia ou Thunder Client.

Faça:

POST `http://localhost:8080/auth/login`

Body:

```
{  
  "username": "admin",  
  "password": "123456"  
}
```

Resposta:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9..."  
}
```

Esse token será utilizado nas próximas requisições.

19. Acessando endpoint protegido

Agora:

GET http://localhost:8080/produtos

Sem token:

401 Unauthorized

Com token:

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

o Spring Security permite:

GET /produtos

|

v

JWT Filter

|

v

Extrai token

|

v

Valida assinatura

|

v

Extrai username

|

v

Authentication

|

v

Spring Security

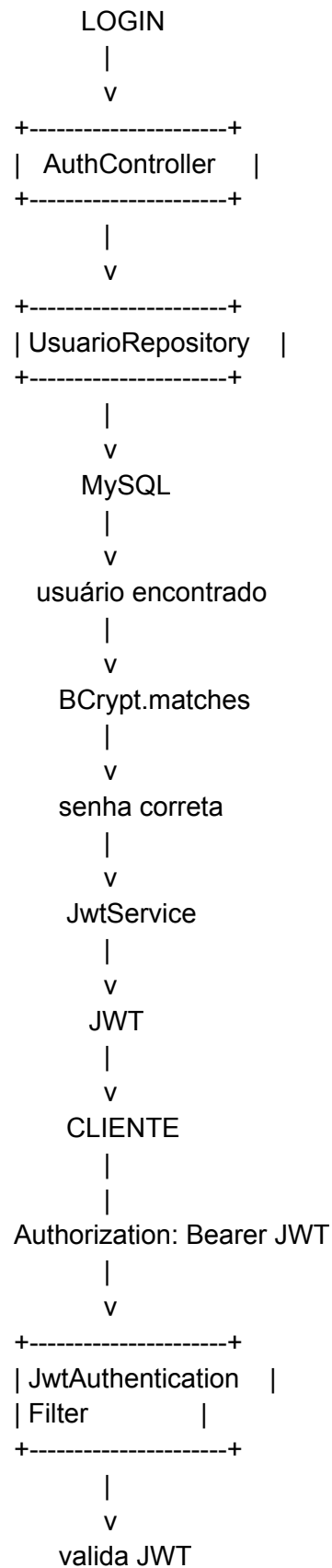
|

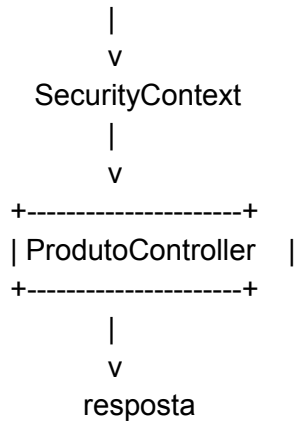
v

ProdutoController

20. O projeto completo conceitualmente

É importante que seus alunos entendam esta sequência:





Uma observação importante

Este exemplo é ótimo para **aprender a arquitetura**, mas há uma melhoria importante para uma aplicação real: o filtro deveria carregar o usuário do banco e suas **roles/authorities** para que possamos fazer, por exemplo:

```
.requestMatchers("/admin/**")
.hasRole("ADMIN")
```

em vez de apenas verificar se o JWT é válido.

Também recomendo não deixar a chave JWT diretamente no código:

```
private final String SECRET = "...";
```

Em produção, ela deve ficar em variável de ambiente/secret manager, e as credenciais do banco também.

Se você quiser montar isso **como projeto didático completo**, o próximo passo natural é eu estruturar uma versão com **cadastro de usuário + login + JWT + BCrypt + MySQL + CRUD de produtos + ROLE_USER/ROLE_ADMIN + Postman**, mostrando **arquivo por arquivo e explicando cada linha**.

Estruture uma versão com cadastro de usuário, login, JWT, BCrypt, MySQL, CRUD de produtos, ROLE_USER/ROLE_ADMIN e Postman

Perfeito. Vamos montar um projeto **completo e didático**, adequado para usar em aula, mostrando o fluxo inteiro:

Cadastro → Login → JWT → Endpoint protegido → Roles → CRUD

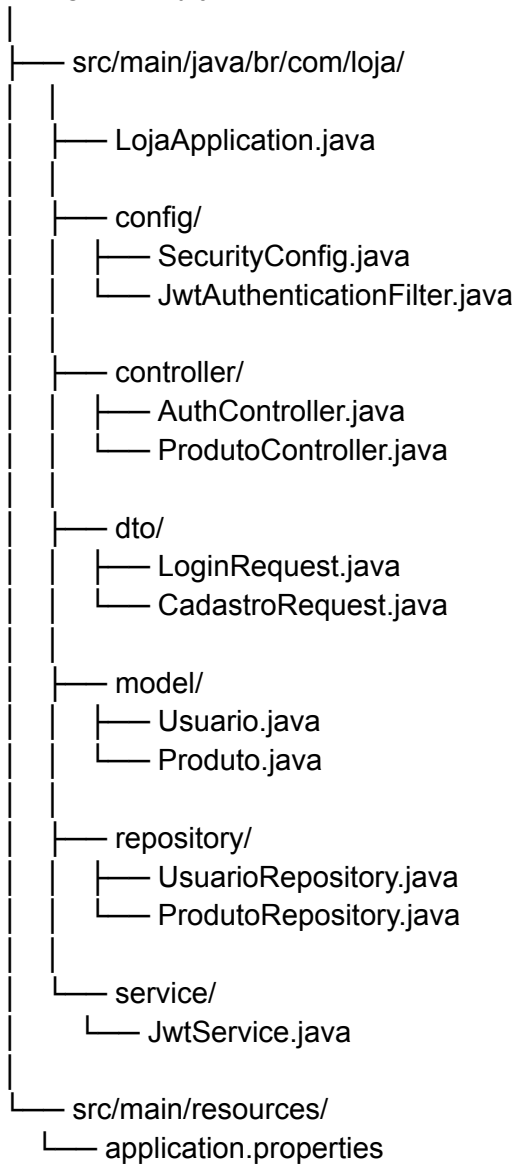
A stack será:

- Java 17+

- Spring Boot
- Spring Security
- JWT
- BCrypt
- Spring Data JPA
- MySQL
- REST API
- Postman

1. Arquitetura do projeto

spring-security-jwt/



2. Banco MySQL

Primeiro crie o banco:

```
CREATE DATABASE loja;
```

```
USE loja;
```

Não precisamos criar as tabelas manualmente. O JPA fará isso.

3. pom.xml

Use:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.13</version>
    <relativePath/>
  </parent>

  <groupId>br.com</groupId>
  <artifactId>loja</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>

    <!-- REST API -->
    <dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Spring Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- MySQL -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

```
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>
        org.springframework.boot
      </groupId>
      <artifactId>
        spring-boot-maven-plugin
      </artifactId>
    </plugin>
  </plugins>
</build>

</project>
```

4. application.properties

spring.application.name=loja

spring.datasource.url=jdbc:mysql://localhost:3306/loja
spring.datasource.username=root
spring.datasource.password=123456

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

server.port=8080

Altere:

spring.datasource.username=root
spring.datasource.password=123456

para o usuário/senha do seu MySQL.

5. Entidade Usuario


```
model/Usuario.java
```

```
package br.com.loja.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "usuarios")
```

```
public class Usuario {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String nome;
```

```
    @Column(unique = true, nullable = false)
```

```
    private String username;
```

```
    @Column(nullable = false)
```

```
    private String password;
```

```
    @Column(nullable = false)
```

```
    private String role;
```

```
    public Usuario() {
```

```
    }
```

```
    public Long getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(Long id) {
```

```
        this.id = id;
```

```
    }
```

```
    public String getNome() {
```

```
        return nome;
```

```
    }
```

```
    public void setNome(String nome) {
```

```
        this.nome = nome;
```

```
    }
```

```

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getRole() {
    return role;
}

public void setRole(String role) {
    this.role = role;
}
}

```

A tabela resultante será aproximadamente:

usuarios

```

-----
id | nome | username | password | role
-----

```

6. Entidade Produto

model/Produto.java

```

package br.com.loja.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

```

@Entity

```
@Table(name = "produtos")
```

```
public class Produto {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String descricao;
```

```
    private BigDecimal preco;
```

```
    private Integer estoque;
```

```
    public Produto() {  
    }
```

```
    public Long getId() {  
        return id;  
    }
```

```
    public void setId(Long id) {  
        this.id = id;  
    }
```

```
    public String getDescricao() {  
        return descricao;  
    }
```

```
    public void setDescricao(String descricao) {  
        this.descricao = descricao;  
    }
```

```
    public BigDecimal getPreco() {  
        return preco;  
    }
```

```
    public void setPreco(BigDecimal preco) {  
        this.preco = preco;  
    }
```

```
    public Integer getEstoque() {  
        return estoque;  
    }
```

```
public void setEstoque(Integer estoque) {  
    this.estoque = estoque;  
}  
}
```

7. Repository de usuário

repository/UsuarioRepository.java

```
package br.com.loja.repository;  
  
import br.com.loja.model.Usuario;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import java.util.Optional;  
  
public interface UsuarioRepository  
    extends JpaRepository<Usuario, Long> {  
  
    Optional<Usuario> findByUsername(String username);  
}
```

O método:

findByUsername()

permitirá procurar o usuário no MySQL.

8. Repository de produto

repository/ProdutoRepository.java

```
package br.com.loja.repository;  
  
import br.com.loja.model.Produto;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface ProdutoRepository  
    extends JpaRepository<Produto, Long> {
```

```
}
```

Com isso já temos:

```
save()
findAll()
findById()
deleteById()
```

9. DTO de cadastro

dto/CadastroRequest.java

```
package br.com.loja.dto;
```

```
public class CadastroRequest {
```

```
    private String nome;
    private String username;
    private String password;
```

```
    public String getNome() {
        return nome;
    }
```

```
    public String getUsername() {
        return username;
    }
```

```
    public String getPassword() {
        return password;
    }
```

```
    public void setNome(String nome) {
        this.nome = nome;
    }
```

```
    public void setUsername(String username) {
        this.username = username;
    }
```

```
    public void setPassword(String password) {
```

```
        this.password = password;
    }
}
```

10. DTO de login

dto/LoginRequest.java

```
package br.com.loja.dto;

public class LoginRequest {

    private String username;
    private String password;

    public String getUsername() {
        return username;
    }

    public String getPassword() {
        return password;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

11. Serviço JWT

service/JwtService.java

```
package br.com.loja.service;

import io.jsonwebtoken.Jwts;
```

```

import io.jsonwebtoken.security.Keys;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;

@Service
public class JwtService {

    private final String SECRET =
        "chave-secreta-muito-grande-para-jwt-123456789";

    private final long EXPIRATION =
        1000 * 60 * 60;

    private SecretKey getKey() {

        return Keys.hmacShaKeyFor(
            SECRET.getBytes()
        );
    }

    public String gerarToken(String username) {

        return Jwts.builder()
            .subject(username)
            .issuedAt(new Date())
            .expiration(
                new Date(
                    System.currentTimeMillis()
                    + EXPIRATION
                )
            )
            .signWith(getKey())
            .compact();
    }

    public String extrairUsername(String token) {

        return Jwts.parser()
            .verifyWith(getKey())
            .build()
            .parseSignedClaims(token)
            .getPayload()
    }

```

```
        .getSubject();  
    }  
}
```

Para aula, a chave pode ficar assim inicialmente. **Em produção, ela deve ficar em variável de ambiente ou secret manager.**

12. Cadastro e Login

Agora o Controller mais importante:

`controller/AuthController.java`

```
package br.com.loja.controller;
```

```
import br.com.loja.dto.CadastroRequest;  
import br.com.loja.dto.LoginRequest;  
import br.com.loja.model.Usuario;  
import br.com.loja.repository.UsuarioRepository;  
import br.com.loja.service.JwtService;
```

```
import org.springframework.http.ResponseEntity;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.Map;
```

```
@RestController
```

```
@RequestMapping("/auth")
```

```
public class AuthController {
```

```
    private final UsuarioRepository usuarioRepository;  
    private final PasswordEncoder passwordEncoder;  
    private final JwtService jwtService;
```

```
    public AuthController(  
        UsuarioRepository usuarioRepository,  
        PasswordEncoder passwordEncoder,  
        JwtService jwtService) {
```

```
        this.usuarioRepository = usuarioRepository;
```



```
    this.passwordEncoder = passwordEncoder;
    this.jwtService = jwtService;
}
```

```
@PostMapping("/cadastro")
public ResponseEntity<?> cadastro(
    @RequestBody CadastroRequest request) {

    if (usuarioRepository
        .findByUsername(request.getUsername())
        .isPresent()) {

        return ResponseEntity
            .badRequest()
            .body("Usuário já existe");
    }
}
```

```
    Usuario usuario = new Usuario();
```

```
    usuario.setNome(request.getNome());
    usuario.setUsername(request.getUsername());
```

```
    usuario.setPassword(
        passwordEncoder.encode(
            request.getPassword()
        )
    );
```

```
    // Todo novo usuário começa como USER
    usuario.setRole("USER");
```

```
    usuarioRepository.save(usuario);
```

```
    return ResponseEntity.ok(
        "Usuário cadastrado com sucesso"
    );
}
```

```
@PostMapping("/login")
public ResponseEntity<?> login(
    @RequestBody LoginRequest request) {
```

```
    Usuario usuario =
        usuarioRepository
```

```

        .findByUsername(
            request.getUsername()
        )
        .orElse(null);

    if (usuario == null) {

        return ResponseEntity
            .status(401)
            .body("Usuário ou senha inválidos");
    }

    boolean senhaCorreta =
        passwordEncoder.matches(
            request.getPassword(),
            usuario.getPassword()
        );

    if (!senhaCorreta) {

        return ResponseEntity
            .status(401)
            .body("Usuário ou senha inválidos");
    }

    String token =
        jwtService.gerarToken(
            usuario.getUsername()
        );

    return ResponseEntity.ok(
        Map.of(
            "token", token,
            "usuario", usuario.getUsername(),
            "role", usuario.getRole()
        )
    );
}
}

```

Aqui temos dois endpoints:

POST /auth/cadastro

POST /auth/login

Eles são públicos.

13. JWT Filter

Crie:

```
config/JwtAuthenticationFilter.java
```

```
package br.com.loja.config;
```

```
import br.com.loja.service.JwtService;
```

```
import jakarta.servlet.FilterChain;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;
```

```
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;  
import org.springframework.security.core.context.SecurityContextHolder;
```

```
import org.springframework.stereotype.Component;  
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import java.io.IOException;  
import java.util.Collections;
```

```
@Component
```

```
public class JwtAuthenticationFilter  
    extends OncePerRequestFilter {
```

```
    private final JwtService jwtService;
```

```
    public JwtAuthenticationFilter(  
        JwtService jwtService) {
```

```
        this.jwtService = jwtService;  
    }
```

```
@Override
```

```
protected void doFilterInternal(  
    HttpServletRequest request,
```

```

        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

    String header =
        request.getHeader("Authorization");

    if (header == null ||
        !header.startsWith("Bearer ")) {

        filterChain.doFilter(request, response);
        return;
    }

    String token =
        header.substring(7);

    try {

        String username =
            jwtService.extrairUsername(token);

        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                username,
                null,
                Collections.emptyList()
            );

        SecurityContextHolder
            .getContext()
            .setAuthentication(
                authentication
            );

    } catch (Exception e) {

        response.setStatus(
            HttpServletResponse.SC_UNAUTHORIZED
        );

        return;
    }

```

```
        filterChain.doFilter(request, response);
    }
}
```

14. Configuração do Security

Aqui faremos a proteção dos endpoints.

config/SecurityConfig.java

```
package br.com.loja.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
```

```
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

```
@Configuration
```

```
public class SecurityConfig {
```

```
    private final JwtAuthenticationFilter jwtFilter;
```

```
    public SecurityConfig(
        JwtAuthenticationFilter jwtFilter) {
```

```
        this.jwtFilter = jwtFilter;
    }
```

```
@Bean
```

```
public SecurityFilterChain securityFilterChain(
    HttpSecurity http) throws Exception {
```

```
    http
```

```
        // API REST não utiliza formulário CSRF
```

```

.csrf(csrf -> csrf.disable())

// JWT não utiliza sessão
.sessionManagement(session ->
    session.sessionCreationPolicy(
        SessionCreationPolicy.STATELESS
    )
)

.authorizeHttpRequests(auth -> auth

    // Cadastro e login são públicos
    .requestMatchers("/auth/**")
    .permitAll()

    // Somente ADMIN
    .requestMatchers("/admin/**")
    .hasRole("ADMIN")

    // USER ou ADMIN
    .requestMatchers("/produtos/**")
    .hasAnyRole("USER", "ADMIN")

    // Qualquer outra URL
    .anyRequest()
    .authenticated()
)

.addFilterBefore(
    jwtFilter,
    UsernamePasswordAuthenticationFilter.class
);

return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() {

    return new BCryptPasswordEncoder();
}
}

```

15. Um detalhe importante: ROLE

No banco armazenaremos:

USER
ADMIN

Quando utilizamos:

```
.hasRole("ADMIN")
```

o Spring Security trabalha internamente com:

ROLE_ADMIN

Da mesma forma:

```
.hasAnyRole("USER", "ADMIN")
```

representa:

ROLE_USER
ROLE_ADMIN

16. CRUD de produtos

Agora o Controller:

```
controller/ProdutoController.java
```

```
package br.com.loja.controller;
```

```
import br.com.loja.model.Produto;  
import br.com.loja.repository.ProdutoRepository;
```

```
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController  
@RequestMapping("/produtos")
```

```

public class ProdutoController {

    private final ProdutoRepository repository;

    public ProdutoController(
        ProdutoRepository repository) {

        this.repository = repository;
    }

    // CREATE
    @PostMapping
    public Produto salvar(
        @RequestBody Produto produto) {

        return repository.save(produto);
    }

    // READ
    @GetMapping
    public List<Produto> listar() {

        return repository.findAll();
    }

    // READ ONE
    @GetMapping("/{id}")
    public ResponseEntity<Produto> consultar(
        @PathVariable Long id) {

        return repository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(
                ResponseEntity.notFound().build()
            );
    }

    // UPDATE
    @PutMapping("/{id}")
    public ResponseEntity<Produto> alterar(
        @PathVariable Long id,
        @RequestBody Produto produto) {

        return repository.findById(id)

```



```

        .map(p -> {

            p.setDescricao(
                produto.getDescricao()
            );

            p.setPreco(
                produto.getPreco()
            );

            p.setEstoque(
                produto.getEstoque()
            );

            return ResponseEntity.ok(
                repository.save(p)
            );
        })
        .orElse(
            ResponseEntity.notFound().build()
        );
    }

// DELETE
@DeleteMapping("/{id}")
public ResponseEntity<Void> excluir(
    @PathVariable Long id) {

    if (!repository.existsById(id)) {

        return ResponseEntity
            .notFound()
            .build();
    }

    repository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}

```

17. Vamos melhorar as permissões

Para demonstrar realmente USER e ADMIN, podemos decidir:

	USER	ADMIN

GET produtos	✓	✓
POST produto	✓	✓
PUT produto	✓	✓
DELETE produto	✗	✓

Para isso, altere a configuração:

```
.requestMatchers(HttpMethod.DELETE, "/produtos/**")  
.hasRole("ADMIN")
```

```
.requestMatchers("/produtos/**")  
.hasAnyRole("USER", "ADMIN")
```

Precisamos importar:

```
import org.springframework.http.HttpMethod;
```

Então o trecho fica:

```
.authorizeHttpRequests(auth -> auth
```

```
    .requestMatchers("/auth/**")  
    .permitAll()
```

```
    .requestMatchers(  
        HttpMethod.DELETE,  
        "/produtos/**"  
    )  
    .hasRole("ADMIN")
```

```
    .requestMatchers("/produtos/**")  
    .hasAnyRole("USER", "ADMIN")
```

```
    .anyRequest()  
    .authenticated()  
)
```

Agora temos uma aplicação com autorização de verdade.

18. Criando um usuário USER pelo Postman

No Postman:

POST `http://localhost:8080/auth/cadastro`

Body → raw → JSON:

```
{
  "nome": "João",
  "username": "joao",
  "password": "123456"
}
```

Resposta:

Usuário cadastrado com sucesso

No banco:

id | nome | username | password | role

teremos algo como:

1 | João | joao | \$2a\$10\$... | USER

Observe:

123456

não aparece no banco.

19. Login

Agora:

POST `http://localhost:8080/auth/login`

Body:

```
{
  "username": "joao",
  "password": "123456"
}
```

Resposta:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "usuario": "joao",
  "role": "USER"
}
```

Copie o token.

20. Acessando o CRUD

No Postman:

GET <http://localhost:8080/produtos>

Na aba **Authorization**:

Type: Bearer Token

Token:

eyJhbGciOiJIUzI1NiJ9...

Ou manualmente em Headers:

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Agora:

GET /produtos

será autorizado.

21. Cadastrando produto

POST http://localhost:8080/produtos

Authorization:

Bearer TOKEN

Body:

```
{
  "descricao": "Notebook",
  "preco": 3500.00,
  "estoque": 10
}
```

Resposta:

```
{
  "id": 1,
  "descricao": "Notebook",
  "preco": 3500.00,
  "estoque": 10
}
```

22. Alterando produto

PUT http://localhost:8080/produtos/1

Body:

```
{
  "descricao": "Notebook Dell",
  "preco": 4200.00,
  "estoque": 15
}
```

23. Consultando produto

GET http://localhost:8080/produtos/1

Resposta:

```
{
```

```
"id": 1,  
"descricao": "Notebook Dell",  
"preco": 4200.00,  
"estoque": 15  
}
```

24. Excluindo produto

Agora vem o teste de autorização.

Usuário:

```
joao  
ROLE_USER
```

faz:

```
DELETE http://localhost:8080/produtos/1
```

Resultado:

403 Forbidden

Porque:

```
.hasRole("ADMIN")
```

é exigido para DELETE.

25. Criando ADMIN

O cadastro público **não deve permitir que o usuário escolha a própria role.**

Por exemplo, não devemos receber:

```
{  
  "nome": "João",  
  "username": "joao",  
  "password": "123456",  
  "role": "ADMIN"
```

```
}
```

Isso seria uma falha de segurança.

O nosso DTO nem possui `role`.

Portanto:

```
usuario.setRole("USER");
```

sempre cria usuário comum.

Para criar o administrador, podemos inicialmente alterar diretamente no banco:

```
UPDATE usuarios  
SET role = 'ADMIN'  
WHERE username = 'admin';
```

Mas primeiro precisamos ter esse usuário.

Faça o cadastro:

```
POST /auth/cadastro  
{  
  "nome": "Administrador",  
  "username": "admin",  
  "password": "123456"  
}
```

Depois:

```
UPDATE usuarios  
SET role = 'ADMIN'  
WHERE username = 'admin';
```

26. Login do administrador

```
POST http://localhost:8080/auth/login  
{  
  "username": "admin",  
  "password": "123456"  
}
```

Resposta:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "usuario": "admin",
  "role": "ADMIN"
}
```

Use esse token:

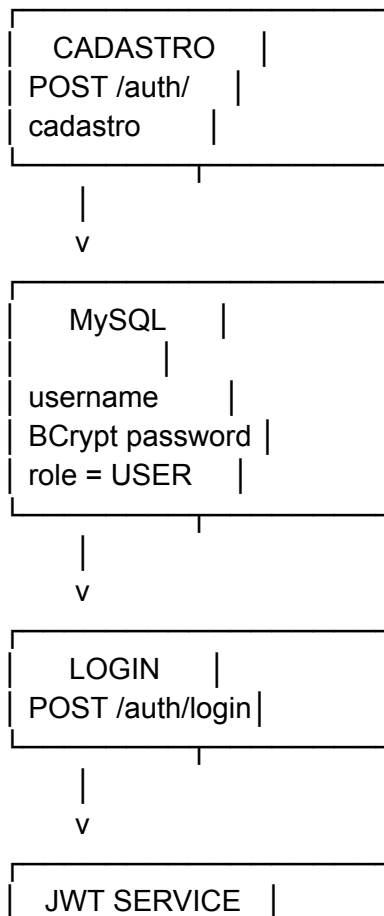
DELETE /produtos/1

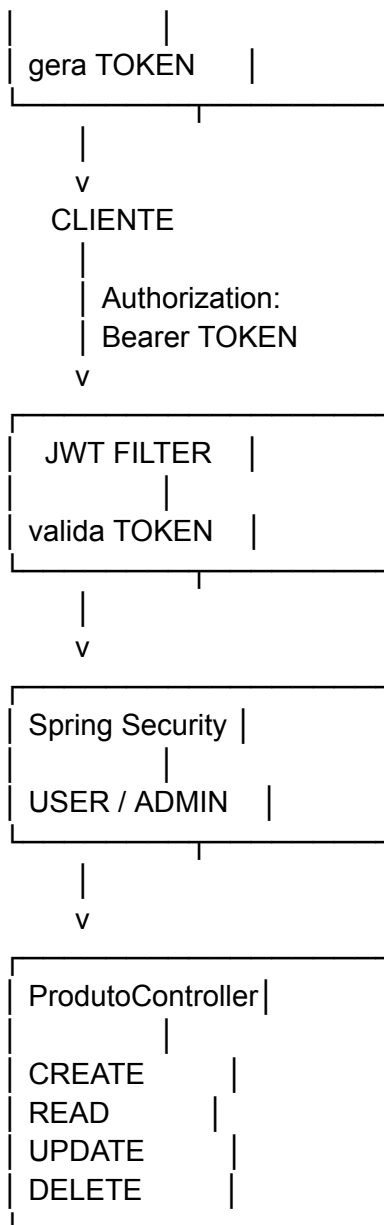
Agora:

204 No Content

27. Fluxo completo para o aluno

Eu apresentaria o projeto em aula desta maneira:





28. Mapa dos endpoints

Método	Endpoint	Autenticação	Permissão
POST	/auth/cadastro	Não	Público
POST	/auth/login	Não	Público
GET	/produtos	Sim	USER/ADMIN

GET	/produtos/{ id}	Sim	USER/ADMIN
POST	/produtos	Sim	USER/ADMIN
PUT	/produtos/{ id}	Sim	USER/ADMIN
DELETE	/produtos/{ id}	Sim	ADMIN